

RELATÓRIO - TRABALHO 2 - COMPACTADOR E DESCOMPACTADOR

Disciplina: INF1010 - Estruturas de Dados (EDA) **Alunos:** Gabriel Adib - 2510466; Gabriel Félix - 2510619

Tema: Compactação e descompactação de texto utilizando códigos de Huffman.

Objetivo

Implementar e testar um programa em linguagem C para compactar e descompactar um arquivo de texto, utilizando uma estrutura inspirada no algoritmo de Huffman. O trabalho tem como finalidade calcular a frequência dos caracteres, construir uma árvore de codificação, gerar códigos binários para os caracteres encontrados, compactar o texto original e, posteriormente, gerar uma versão descompactada para comparação.

Estrutura do Programa

Arquivo	Descrição
huffman.h	Define as constantes e as estruturas principais. A estrutura No representa um nó da árvore, com caractere, frequência e ponteiros para os filhos esquerdo e direito. TabelaFreq armazena o caractere e sua frequência. TabelaCodigo armazena o caractere, o código binário e seu tamanho. Também declara as funções contarFrequencia, criarArvore, gerarCodigos, compactar e descompactar.
huffman.c	Implementa as funções do trabalho. Conta a frequência dos caracteres do arquivo de entrada, cria os nós da árvore, gera a tabela de códigos por percurso recursivo, grava o arquivo compactado com sequências de 0 e 1 e realiza a leitura desse arquivo para reconstruir o texto descompactado.
main.c	Controla a execução. Define o arquivo textoCompactadorDescompactador.txt como entrada, imprime a tabela de frequências, cria a árvore, gera e imprime a tabela de códigos de Huffman, chama a compactação para compactado.txt e a descompactação para descompactado.txt.

Solução / Lógica Utilizada

Inicialmente, o programa abre o arquivo de texto de entrada e percorre todos os caracteres, preenchendo um vetor de frequências de tamanho 256, correspondente aos caracteres da tabela ASCII. Em seguida, a função criarArvore cria nós para os caracteres presentes no texto. A partir dessa árvore, a função gerarCodigos percorre os ramos, acrescentando 0 para a esquerda e 1 para a direita, formando a tabela de códigos binários.

Na compactação, o arquivo original é lido caractere por caractere. Para cada caractere encontrado, o programa procura seu código na tabela de Huffman e grava a sequência correspondente no arquivo compactado.txt. Na descompactação, o arquivo compactado é lido bit a bit, percorrendo a árvore até alcançar um nó folha. Quando isso ocorre, o caractere é escrito em descompactado.txt, e o percurso retorna para a raiz.

Saída e Testes Realizados

O main.c imprime a tabela de frequências dos caracteres encontrados no texto e, em seguida, a tabela de códigos gerados. Depois, o programa cria os arquivos compactado.txt e descompactado.txt, exibindo uma mensagem de finalização do processo. Para compilar, utiliza-se: gcc main.c huffman.c -o trab2. Para executar, utiliza-se: ./trab2.

Os testes verificam se o programa lê corretamente o arquivo textoCompactadorDescompactador.txt, calcula as frequências, gera os códigos, grava o arquivo compactado e cria o arquivo descompactado. Assim, é possível comparar o texto original com o resultado final da descompactação.

Observações e Conclusão

O programa está organizado em módulos e executa as etapas principais esperadas de um compactador/descompactador: cálculo de frequências, criação da árvore, geração de códigos, compactação e descompactação. Como limitação, a construção da árvore está simplificada e não utiliza uma fila de prioridade, que seria a forma clássica do algoritmo de Huffman. Além disso, o arquivo compactado é salvo como texto contendo 0 e 1, e não como bits reais. Mesmo assim, o trabalho demonstra corretamente a ideia geral de codificação por frequência e o uso de árvores em Estruturas de Dados.